# Atividade: Comparação de desempenho entre métodos de ordenação

## Contexto

No encontro anterior, a turma observou como as pessoas organizam números à mão e associou essas estratégias aos algoritmos clássicos. Agora a escala muda.

Dez papéis numerados em cima da mesa escondem a diferença entre os métodos, porque qualquer um deles resolve o problema em poucos segundos. A diferença só aparece no volume. Com dez mil números, um algoritmo termina em menos de um milissegundo, enquanto outro executa dezenas de milhões de comparações para chegar ao mesmo resultado.

Nesta atividade você observa os cinco algoritmos no simulador de ordenação do escalada.dev, constrói um único programa em C que executa os cinco sobre os mesmos dados e mede o desempenho de cada um em três cenários. No final, com os números do seu programa, você escreve o parecer sobre qual método é o melhor.

A atividade é individual. Cada aluno constrói o próprio programa, coleta os próprios números e escreve a própria análise. Não existe gabarito para copiar da internet, porque os números são os da sua execução.

## O que você vai entregar

São dois arquivos, e os dois são seus:

- o programa em C, em um único arquivo chamado `nome_sobrenome_ordenacao.c`, com os cinco algoritmos, os contadores, os três cenários, a medição de tempo e o resumo final;
- o documento em .DOC ou .DOCX, com o relato da observação, a validação dos contadores, as três tabelas, o resumo impresso pelo programa e o seu parecer.

## Objetivos

- Observar, na prática, a estratégia de cada um dos cinco métodos de ordenação.
- Identificar, no código de cada método, onde acontecem a comparação, a troca e a movimentação.

- Medir comparações, trocas, movimentações e tempo de execução com dez mil números.
- Verificar como o estado inicial dos dados afeta o desempenho de cada método.
- Relacionar o comportamento medido à notação O estudada em aula.
- Emitir um parecer técnico sobre qual algoritmo usar, justificado pelos números.

## Prazo e formato de entrega

- Data de início:
- Data de entrega:
- Canal de entrega:
- Arquivos: o programa .c e o documento .DOC ou .DOCX, com o seu nome completo na primeira página.

O trabalho é individual do começo ao fim. Conversar sobre as ideias com colegas ajuda, mas o programa, as tabelas e o parecer entregues precisam ser seus.

## Os quatro indicadores

O programa inteiro gira em torno de quatro números. Antes de escrever código, é preciso saber exatamente o que cada um conta, porque um contador no lugar errado produz uma tabela bonita e uma análise errada.

### Comparação

Uma comparação acontece sempre que dois valores do vetor são comparados para decidir a ordem. Por exemplo:

```
if(v[i] > v[i + 1])
```

A comparação conta mesmo quando o resultado é falso. Os testes que só controlam a repetição, como `i < TAM` no `for` ou `j >= 0` no `while`, não comparam dados e não entram no contador.

### Troca

Uma troca acontece quando dois elementos trocam de posição entre si. Cada troca completa conta uma vez:

```
aux = v[i];
v[i] = v[j];
v[j] = aux;
```

Trocar uma posição com ela mesma não conta, porque nenhum valor muda de lugar. Isso acontece no Quick Sort quando a função `troca` recebe duas vezes o mesmo endereço.

## Movimentação

Uma movimentação é cada atribuição que leva um valor do vetor para outro lugar: outra posição do vetor, a variável auxiliar, a chave, o pivô ou o vetor temporário. No trecho de troca acima há 1 troca e 3 movimentações, porque são três atribuições de valor.

Troca e movimentação não são a mesma coisa. Toda troca gera três movimentações, mas nem toda movimentação vem de uma troca.

## Algoritmos que não fazem troca

O Insertion Sort e o Merge Sort ordenam sem trocar dois elementos de posição. O Insertion Sort guarda o valor na chave, desloca os maiores para a direita e insere a chave no lugar certo. O Merge Sort copia os valores para um vetor temporário e depois de volta para o vetor. Nos dois, o contador de trocas termina em zero, e o trabalho aparece nas movimentações.

Zero trocas no Insertion Sort e no Merge Sort é o resultado correto, e não defeito do programa. Na análise, esses dois métodos são avaliados pelas comparações, pelas movimentações e pelo tempo.

## Tempo de execução

O tempo mede somente a ordenação. A geração do vetor, a cópia, a impressão e a montagem das tabelas ficam fora da medição. Em C ANSI, use a biblioteca `time.h`:

```
#include <time.h>

clock_t inicio, fim;
double tempo_ms;

inicio = clock();
/* chamada da ordenação, por exemplo bubbleSort(vetorTeste,
TAM); */
fim = clock();

tempo_ms = (double)(fim - inicio) * 1000.0 / CLOCKS_PER_SEC;
```

O relógio do computador não enxerga intervalos muito curtos. Um método rápido pode aparecer com 0 ms, o que significa que ele terminou antes da menor fração que o relógio consegue medir.

Registre na tabela o valor que o programa mostrou e comente isso na análise.

## Onde cada indicador aparece em cada método

O quadro mostra em que ponto do código de cada método os indicadores acontecem. Os códigos são os do material de apoio Modelos de Ordenação. Cabe a você colocar os contadores nesses pontos e comentar, no programa, o que cada um conta.

### Onde estão a comparação, a troca e a movimentação

| Método | Comparação | Troca | Movimentação |
|---|---|---|---|
| **Bubble Sort** | `v[i] > v[i + 1]`, entre vizinhos | quando o par está fora de ordem | 3 por troca |
| **Selection Sort** | `v[j] < v[min]`, na busca do menor | uma por passagem, só quando `min != i` | 3 por troca |
| **Insertion Sort** | `v[j] > chave`, dentro do `while` | não tem | guardar a chave, cada deslocamento e a inserção |
| **Merge Sort** | `v[i] <= v[j]`, na intercalação | não tem | cada cópia para `temp` e cada cópia de volta para `v` |
| **Quick Sort** | `v[j] < pivo`, na partição | cada chamada de `troca` com posições diferentes | guardar o pivô e 3 por troca |

No `while` do Insertion Sort, a condição tem duas partes. O teste `j >= 0` é controle, e o teste `v[j] > chave` é comparação: o contador soma só quando esse segundo teste chega a ser feito.

## Contagens que seguem outra regra

Outros materiais contam de outro jeito, e os números deles não servem de comparação aqui. Os gráficos do livro Estruturas de Dados, de Ana Fernanda Gomes Ascencio, usados em semestres anteriores, são um exemplo. Nesta atividade vale a regra do simulador, e estas são as diferenças que mais aparecem:

- No Selection Sort, o contador de trocas fica dentro do `if(min != i)`. Fora dele, o programa soma uma troca por passagem mesmo quando nenhum valor muda de lugar, e o vetor já ordenado passa a mostrar 9.999 trocas em vez de 0.
- No Insertion Sort, o `while` pode terminar porque `j` chegou a -1, e nesse caso a comparação `v[j] > chave` não acontece. Somar uma comparação fixa a cada passagem, além das feitas no `while`, conta comparações que não existiram, e o vetor invertido passa a mostrar 50.004.999 comparações em vez de 49.995.000.

- No Insertion Sort e no Merge Sort, deslocar um valor ou copiá-lo para o vetor temporário é movimentação, e não troca. Chamar isso de troca esconde justamente a diferença que a análise precisa mostrar.
- No Merge Sort, a cópia de volta, do vetor temporário para o vetor, também é movimentação e entra no contador.
- No tempo, a conversão para milissegundos usa `CLOCKS_PER_SEC`, e não um número fixo, porque o valor dessa constante muda de um compilador para outro.

## Etapa 1 - Observação no simulador

Antes de programar, é preciso entender o que cada método faz. Abra o simulador de ordenação em escalada.dev/modelos/estrutura-de-dados/ordenacao/, escolha 8 cartas e execute os cinco algoritmos, com o botão Reproduzir ou passo a passo, com o botão Próximo.

Trocar de método mantém a mesma distribuição de cartas. Assim os cinco métodos trabalham sobre os mesmos valores, do mesmo jeito que vão trabalhar no seu programa.

Anote a sequência inicial das cartas. Para cada método, escreva de três a cinco linhas descrevendo o que você viu acontecer na tela, com as suas palavras. Não pesquise a definição do algoritmo nesta etapa: o objetivo é o relato da observação.

Perguntas que ajudam a organizar o relato:

- Quais elementos o algoritmo compara a cada passo?
- Em que momento ele faz uma troca?
- Em que momento ele movimenta um valor sem trocar?
- Onde a parte já ordenada vai se formando, no início ou no fim?
- O algoritmo percorre o vetor inteiro várias vezes ou trabalha por partes?

Ao final de cada método, anote os três contadores que o simulador mostra: comparações, trocas e movimentações. Eles são usados na validação do seu programa.

## Etapa 2 - O programa único em C

Agora você constrói um único programa em C ANSI que executa os cinco métodos, mede os quatro indicadores e mostra o resultado. Não são cinco programas separados: é um arquivo só. Os códigos do material de apoio Modelos de Ordenação podem ser o ponto de partida, e não é preciso escrever os algoritmos do zero.

Use as versões do material de apoio. Outras versões dos mesmos métodos, como o Quick Sort com o pivô no início ou no meio do vetor, contam de outro jeito e não conferem com o simulador na validação.

O programa precisa:

1. conter as funções dos cinco métodos: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort e Quick Sort;
2. ter contadores de comparações, trocas e movimentações, zerados antes de cada execução;
3. gerar os três cenários com 10.000 elementos;
4. entregar a cada método uma cópia do mesmo vetor inicial;
5. medir o tempo de cada ordenação em milissegundos;
6. imprimir uma tabela por cenário;
7. imprimir, no final, o resumo com o melhor método em cada indicador.

## Validação dos contadores

Antes de medir com 10.000 elementos, confira se os contadores estão no lugar certo. Faça o programa ordenar as mesmas 8 cartas da Etapa 1, na mesma ordem inicial, e compare os números com os do simulador. O simulador conta pelas mesmas regras desta atividade, então os cinco métodos precisam dar os mesmos valores.

Se algum número divergir, o contador daquele método está no lugar errado ou está faltando. Corrija e rode de novo até os cinco conferirem.

Sequência inicial das 8 cartas:

## Conferência entre o simulador e o programa (C = comparações, T = trocas, M = movimentações)

| Método | Simulador: C, T e M | Programa: C, T e M | Os números conferem? |
|---|---|---|---|
| **Bubble Sort** | | | |
| **Selection Sort** | | | |
| **Insertion Sort** | | | |
| **Merge Sort** | | | |
| **Quick Sort** | | | |

## Os três cenários

1. Dados ordenados: vetor com os números de 1 a 10.000 em ordem crescente.
2. Dados invertidos: vetor de 10.000 a 1, em ordem decrescente.
3. Dados aleatórios: vetor com 10.000 números sorteados com `rand() % 10000`, como na função `geraNumero`.

No cenário aleatório, cada método é executado três vezes, e a tabela registra a média. A cada rodada o programa sorteia um vetor novo, mas dentro da mesma rodada os cinco métodos recebem o mesmo vetor. Nos cenários ordenado e invertido, uma execução basta, porque o arranjo inicial é sempre o mesmo.

**O mesmo vetor para todos os métodos**

Esta regra é obrigatória. Um vetor já ordenado por um método não pode ser reaproveitado pelo método seguinte, porque o segundo receberia dados ordenados e a comparação perderia o sentido. Trabalhe com dois vetores: o `vetorBase`, criado uma vez por cenário, e o `vetorTeste`, que recebe uma cópia do `vetorBase` antes de cada método.

```
vetorBase
    |
    +-- cópia --> Bubble Sort
    |
    +-- cópia --> Selection Sort
    |
    +-- cópia --> Insertion Sort
    |
    +-- cópia --> Merge Sort
    |
    +-- cópia --> Quick Sort
```

**Organização sugerida**

```
main()
 |
 +-- para cada cenário:
 |     +-- geraNumero(vetorBase, op)
 |     |     op 1: ordenado   op 2: invertido   op 3: aleatório
 |     +-- para cada método:
 |     |     +-- copiar vetorBase para vetorTeste
 |     |     +-- zerar os contadores
 |     |     +-- medir o tempo só da ordenação
 |     |     +-- guardar comparações, trocas, movimentações e
tempo
 |     +-- no aleatório, repetir três vezes e calcular a média
 |     +-- imprimir a tabela do cenário
 |
```

```
 +-- imprimir o resumo
```

Essa organização é só uma referência. O programa pode ser organizado de outro jeito, desde que atenda todos os requisitos.

## Saída esperada

A saída mostra uma tabela por cenário e, no final, o resumo, por exemplo:

```
============================================================
CENARIO 1 - DADOS ORDENADOS - 10000 ELEMENTOS
============================================================
Metodo       Comparacoes    Trocas    Movimentacoes    Tempo
(ms)
Bubble
Selection
Insertion
Merge
Quick

(o mesmo para o cenario 2, dados invertidos, e para o
cenario 3, dados aleatorios, com a media de 3 execucoes)

============================================================
RESUMO
============================================================
Cenario      Menos comparacoes   Menos movimentacoes   Menor
tempo
Ordenados
Invertidos
Aleatorios
```

O resumo não aponta o melhor por trocas. O Insertion Sort e o Merge Sort marcam zero trocas porque não trocam elementos, e não porque trabalham menos, e eleger o melhor por trocas colocaria os dois na frente sem motivo. Quando dois métodos empatarem em um indicador, o resumo mostra os dois.

O formato não precisa ser exatamente este, mas todas as informações precisam estar identificadas.

## Identificação do código-fonte

O arquivo .c começa com este comentário:

```
/*
Disciplina: Estrutura de Dados
Atividade: Comparacao de desempenho entre metodos de ordenacao
```

```
Aluno:

Conseguiu chegar nos valores de referencia?
R:
Se nao, em quais metodos e por que?
R:

Metodos analisados:
- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
*/
```

## Tabelas para registro dos resultados

Copie para o documento os números que o seu programa mostrou.

## Cenário 1 - Dados ordenados (10.000 elementos)

| Método | Comparações | Trocas | Movimentações | Tempo (ms) |
|---|---|---|---|---|
| **Bubble Sort** | | | | |
| **Selection Sort** | | | | |
| **Insertion Sort** | | | | |
| **Merge Sort** | | | | |
| **Quick Sort** | | | | |

## Cenário 2 - Dados invertidos (10.000 elementos)

| Método | Comparações | Trocas | Movimentações | Tempo (ms) |
|---|---|---|---|---|
| **Bubble Sort** | | | | |
| **Selection Sort** | | | | |
| **Insertion Sort** | | | | |
| **Merge Sort** | | | | |
| **Quick Sort** | | | | |

## Cenário 3 - Dados aleatórios (10.000 elementos, média de três execuções)

| Método | Comparações | Trocas | Movimentações | Tempo (ms) |
|---|---|---|---|---|
| **Bubble Sort** | | | | |
| **Selection Sort** | | | | |
| **Insertion Sort** | | | | |
| **Merge Sort** | | | | |
| **Quick Sort** | | | | |

## Valores de referência

Estes são os números que o seu programa deve mostrar, se ele usa os códigos do material de apoio e conta pelas regras desta atividade. Eles foram obtidos com um programa construído dessa forma. Use-os para saber se você conseguiu chegar no resultado.

Nos dados ordenados e invertidos o vetor é sempre o mesmo, e os números precisam ser exatamente estes:

## Cenário 1 - Dados ordenados

| Método | Comparações | Trocas | Movimentações |
|---|---|---|---|
| **Bubble Sort** | 9.999 | 0 | 0 |
| **Selection Sort** | 49.995.000 | 0 | 0 |
| **Insertion Sort** | 9.999 | 0 | 19.998 |
| **Merge Sort** | 69.008 | 0 | 267.232 |
| **Quick Sort** | 49.995.000 | 0 | 9.999 |

## Cenário 2 - Dados invertidos

| Método | Comparações | Trocas | Movimentações |
|---|---|---|---|
| **Bubble Sort** | 99.990.000 | 49.995.000 | 149.985.000 |
| **Selection Sort** | 49.995.000 | 5.000 | 15.000 |
| **Insertion Sort** | 49.995.000 | 0 | 50.014.998 |
| **Merge Sort** | 64.608 | 0 | 267.232 |
| **Quick Sort** | 49.995.000 | 5.000 | 24.999 |

Nos dados aleatórios o vetor muda a cada sorteio, e o número exato também. A média das suas três execuções deve cair dentro destas faixas, medidas em 30 sorteios com `rand() % 10000`. Dois valores não mudam com o sorteio: as comparações do Selection Sort e as movimentações do Merge Sort.

## Cenário 3 - Dados aleatórios, faixa esperada

| Método | Comparações | Trocas | Movimentações |
|---|---|---|---|
| **Bubble Sort** | entre 97 e 100 milhões | entre 24,6 e 25,4 milhões | entre 73,9 e 76,1 milhões |
| **Selection Sort** | 49.995.000, sempre | perto de 9.990 | perto de 29.970 |
| **Insertion Sort** | entre 24,6 e 25,4 milhões | 0 | entre 24,6 e 25,4 milhões |
| **Merge Sort** | entre 120 mil e 121 mil | 0 | 267.232, sempre |
| **Quick Sort** | entre 140 mil e 165 mil | entre 60 mil e 90 mil | entre 190 mil e 270 mil |

O tempo não tem valor de referência, porque depende do computador. O que se espera dele é a mesma ordem entre os métodos que os contadores mostram.

Se um número não bateu, volte à validação com as 8 cartas e confira o contador daquele método. A diferença entre o seu valor e o de referência costuma apontar o erro: uma troca por passagem a mais, uma comparação a mais por passagem, ou uma cópia que ficou fora do contador.

## Análise e parecer

Responda com os números da sua execução. Resposta genérica, sobre os algoritmos em geral, não é suficiente.

1. Qual método fez menos comparações em cada cenário? Cite os valores.

2. Qual método fez mais comparações? O resultado foi o mesmo nos três cenários?

3. Por que o Insertion Sort e o Merge Sort registraram zero trocas? Onde o trabalho desses dois métodos aparece na sua tabela?

4. Qual método teve mais movimentações? Explique por que troca e movimentação não são a mesma operação.

5. O tempo acompanhou os contadores? Algum método foi mais rápido ou mais lento do que os números de comparações e movimentações faziam esperar?

6. Qual método mudou mais de comportamento entre os dados ordenados, invertidos e aleatórios? Mostre os números.

7. Os resultados conferem com a notação O estudada em aula?

## Parecer

Feche a análise com o seu parecer: para ordenar um grande volume de dados, em condições parecidas com as que você testou, qual algoritmo você escolheria?

O parecer precisa citar pelo menos três indicadores entre comparações, movimentações, tempo e comportamento nos três cenários. Informar só o nome do algoritmo não basta. Se o método escolhido não faz trocas, diga isso e justifique a escolha pelas movimentações.

## Estrutura da entrega

O documento entregue deve conter, nesta ordem:

1. Identificação, com o seu nome completo.
2. Etapa 1: relato da observação dos cinco métodos no simulador.
3. Validação: a sequência das 8 cartas e o quadro de conferência entre o simulador e o programa.
4. As três tabelas preenchidas e a resposta: conseguiu chegar nos valores de referência? Se não, em quais métodos e por quê?
5. O resumo impresso pelo programa.
6. As respostas da análise e o parecer.

O programa .c vai junto, em arquivo separado.

## Critérios de avaliação

- Programa único em C que compila, executa os cinco métodos nos três cenários e entrega o mesmo vetor inicial a todos.
- Contadores de comparação, troca e movimentação no lugar certo, conferidos com o simulador e com os valores de referência.
- Tempo medido somente sobre a ordenação.
- Tabelas completas e resumo impresso pelo programa.
- Relato de observação escrito com as próprias palavras, coerente com o que o simulador mostra.
- Parecer apoiado nos números medidos, e não em texto genérico sobre os algoritmos.

Código que não compila, ou que não executa os três cenários, não recebe pontuação integral.